Chapter route

Which computing needs and constraints drove OS evolution, and how do historical designs relate to today's abstractions?

Prerequisites are OS fundamentals, process/memory/file abstractions, and architecture history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Compare historical systems by requirements/constraints.
- Distinguish inherited concepts from version-specific mechanisms.
- Avoid equating historical products with current systems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Feature migration

Compare constraints, mechanisms, and new contracts. Similar names do not establish identical address units, execution agents, or protection boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Concepts migrate without identical implementations

The chapter follows constraint→mechanism→contract, not a strict chronology. Batch sequencing reduces operator waits;spooling decouples device producers/consumers;time sharing adds interaction/CPU reclamation;virtual memory separates information identity from residence.

Constraint	Mechanism	New obligations
Slow manual job changes	Resident monitor	Detect termination/protect monitor
Slow cards/printing	Offline I/O, spool	Queues, capacity, errors/retries
Long exclusive occupancy	Time sharing	Timers, isolation, response, fairness
Expensive fast memory	Paging/backing	Restartable faults, mapping, replacement

Multics, UNIX, and smaller successors share multiuser, hierarchical-file, and sharing concepts. Direct code descent requires source/design evidence beyond conceptual similarity.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Four comparison dimensions

A process can mean static worker, dynamically created process, or a Mach task's thread. Paging can mean relocation without demand loading. Identify:

1. Whether execution agents are dynamic and dedicated per job or reused.
2. Byte/word addressing, page size, and table purpose.
3. Software-layer, privilege, address-space, or machine boundaries.
4. Ownership of faults, timeouts, buffers, and retry/release conditions.

Different answers prohibit transferring modern API/performance promises solely by name. Numerical demonstrations specify rules rather than claim historical measurements.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Early systems

Batch/monitors reduce manual waits;spooling overlaps device work. One printer remains bounded by its service demand.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From manual operation to resident monitors

Compilation, assembly, loading, running required media changes/termination observation. Batching groups similar needs;monitor control records choose next steps. Controllers/drivers still perform device operations.

Three jobsF, C, F each need2 CPU minutes. Each compiler setup costs 3, including initial setup. Original order has9 setup+6 CPU=15. ReorderingF, F, C has6+6=12, assuming reorderability and excluding response/dependencies.

Protect the monitor against user overwrite or halt. Control-record/data separation needs syntax, length, and escaping rules rather than assuming data rarely contains control characters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Offline I/O, buffers, spools, and multiprogramming

Offline equipment converts cards to tape before mainframe processing, saving host waiting but adding media-batch/movement latency. Buffers absorb production/consumption mismatch until finite capacity backpressures. Disk spools retain independent jobs/data/queue metadata for overlap, unlike imagining random simultaneous access to both tape ends. Multiprogramming separately runs another ready job while one waits, and requires scheduling/isolation.

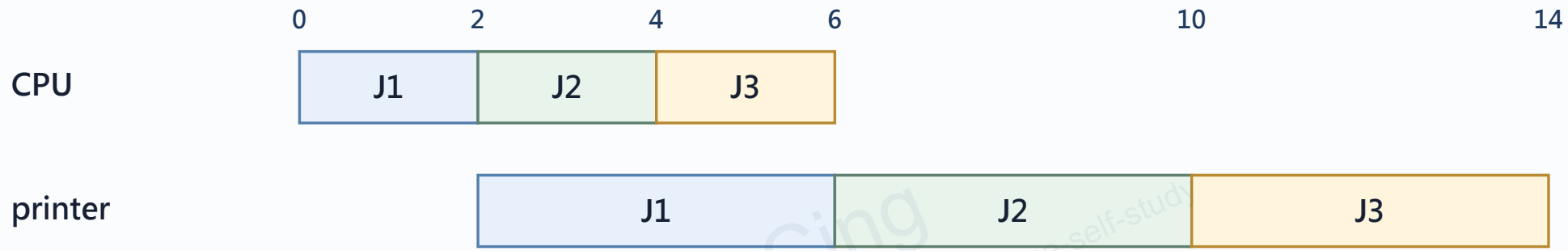
For three jobs each CPU 2 then print4, with one CPU/printer, CPU runsJ1 0–2, J2 2–4, J3 4–6. Printer runsJ1 2–6, J2 6–10, J3 10–14. Total14, CPU utilization6/14, printer 12/14. Serial execution takes 18. Spooling does not halve the printer's12 units of work.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Offline I/O, buffers, spools, and multiprogramming (continued)

Three jobs: CPU 2 each, one printer 4 each; total 14

One color per job. CPU and printer each serialize; different devices overlap.



J2/J3 output can queue. Printer work remains 12. CPU busy 6/14, printer 12/14; serial total 18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Atlas

One-level storage separates information addresses from core/drum residence. Learning replacement includes the all-t=0 fallback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Information identity and automatic backing

Atlas combines fast core/drum behind a one-level store with 512-word pages and associative mappings. The book's16 Kiwords core has32 frames, not16 KiB. A four-frame teaching model retains512 words/page.

Virtual word address3597=7・512+13 maps page 7→frame 2 to core word address1037. If absent, preserve restartable execution, load into an available frame, update mapping, and retry. One-level store does not make drum as fast/directly accessible as core. Transfers, replacement, and live-frame ownership remain necessary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three learning-replacement branches

Kilburn1962 defines t as time since last use and T as the preceding inactive interval, sampled by executed instructions rather than wall time. Choose first any t>T+1;otherwise among t≠0 choose maximalT−t;otherwise all t=0, choose maximalT. This example breaks ties by table order.

Case	(t, T) entries	Branch/victim
A	(6,3),(2,5),(0,9)	First:6>4, first page
B	(2,4),(1,6),(0,8)	Second:nonzero scores2/5, second
C	(0,2),(0,5),(0,3)	Third:largestT5, second

Past intervals predict future use rather than supply OPT knowledge. A predicted-distant page used next can add a fault. Branch3 makes valid all-recently-used states selectable. Original paper.

XDS-940

Paging can relocate without demand paging. Reentrant code can share while writable state remains private.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Relocation is not demand loading

Historical virtual space16 Kiwords, physical64 Kiwords, page 2 Kiwords give eight virtual pages/process and32 frames. Register-held tables scatter resident pages;whole-image swapping changes resident processes.

P virtual page 3→frame 11, offset 17 gives VA6161 and PA11・2048+17=22545 words. No byte conversion or unspecified word width is assumed. A legal missing-page demand fault is additional behavior absent from this relocation-only case. Atlas automatic demand loading and XDS relocation differ despite both having tables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared code and controlled execution

Two processes each use eight pages. Sharing two safely read-only reentrant code pages needs2+6・2=14 frames instead of 16. Shared instruction pages do not privatize a secretly shared writable global buffer;separate code and mutable state.

Converting XDS-930 to940 added user/monitor distinction, privileged I/O/halt, and controlled system calls. Dynamic subprocesses/shared memory additionally require lifecycle, synchronization, termination cleanup. These protections work together.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

THE

Layers and explicit synchronization decompose reasoning. A software layer is not automatically a hardware protection domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lower-layer contracts before upper-layer reasoning

Dijkstra's THE is batch multiprogramming by cooperating sequential processes, not primarily terminal interaction.

Level	Responsibility/hidden details
0	Processor allocation/timer;sequential-process abstraction
1	Segment controller/backing;information-segment identities
2	Console-message interpreter;physical console allocation
3	I/O buffering/logical communication units
4	User programs
5	Operator, not an implemented software level

Layers describe dependency rather than separate address spaces or mandatory traps. Segments identify information;core/drum pages hold it. EWD196.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Induction over layer contracts

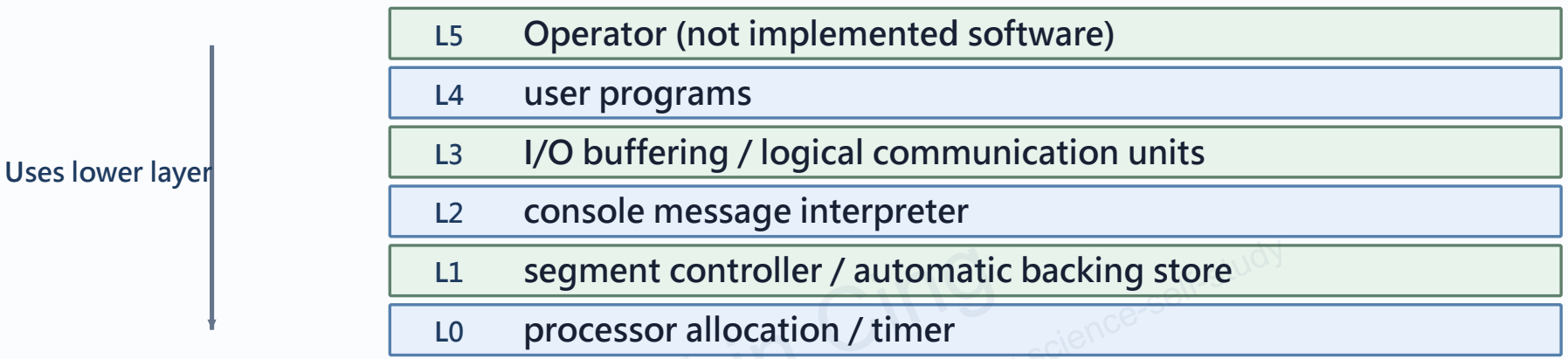
Assume layer i guaranteesCi fromC0…C(i−1), with stated interference constraints, and does not corrupt lower hidden state. Hardware/processor control establishesC0;induction establishes each successive contract. Layering alone proves nothing without those obligations.

Console messages can reside in segments and use level 1 retrieval. If level 1 instead needs level 2 to fetch its own pages, the dependency cycle invalidates bottom-up induction. Synchronization cannot assume relative speed:consume only produced segments, and give a printer only one incompatible owner. Waiting changes progress, not correctness based on "usually slower."

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Induction over layer contracts (continued)

THE: higher levels use lower abstractions, not separate address spaces



Induction: Ci uses/preserves C0…C(i−1). Cycles or uncontrolled shared state invalidate the argument.

https://github.com/xinhi6716/computer-science-self-study

RC4000

A nucleus supplies primitives while upper processes choose policy. Finite buffers require unique ownership and explicit return conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Policy above the nucleus

RC4000 supports constructing different systems rather than hardwiring one batch/time-sharing policy. Nucleus supplies process control, protection, messages. The book's fixed eight-word messages use a shared pool and per-process queues. Send-message/wait-message and send-answer/wait-answer form request/reply. Primitive atomicity protects internal transitions, not an entire server transaction.

Drivers translate registers/interrupts into process-like message interfaces. Devices do not thereby become ordinary processors executing arbitrary user code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conservation with two buffers

Pool={b0, b1};retain one buffer through request/reply completion. This is an ownership model, not instruction-level original-machine emulation.

Step	Free	Server queued/processing	Awaiting client reply
Initial	b0, b1	Empty	Empty
C1 sends	b1	b0 queued	Empty
C2 sends	Empty	b0, b1 queued	Empty
S receives/answers b0	Empty	b1 queued	b0
C1 consumes/returns	b0	b1 queued	Empty

Each buffer has exactly one state;free+queued+processing+reply=2. Third submission must wait/fail as specified rather than allocateb0 again. S waiting for C1 while C1 waits for S still deadlocks. Small kernels/atomic send do not eliminate circular wait.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CTSS

Interaction and swapping shape feedback. Specify initial level, quantum growth, and promotion separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Short interaction versus long CPU bursts

MIT CTSS demonstrates interactive time sharing compatible with existing batch programs. Moving images between scarce memory/backing makes excessively short slices costly. Compatibility concerns preserving that environment, not every future API. The7090/7094 memory counts36-bit words, not equal KiB.

Exponential feedback demotes work exhausting a full slice and gives longer lower-priority slices, while short interactions gain earlier service. I/O promotion, aging, and starting level depend on the chosen version/model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete exponential-feedback trace

Fix q_i=2^i, start0, one burst11, no competitors/promotions;demote only when a full slice expires before completion.

Dispatch	Level	Allowed	Used	Total CPU	Remaining
1	0	1	1	1	10
2	1	2	2	3	8
3	2	4	4	7	4
4	3	8	4	11	0

Full slices throughk sum2^(k+1)−1 by the geometric series. With swap overhead.5/dispatch, four dispatches cost2, total 13. RR q1 needs11 dispatches, overhead5.5, total 16.5. This compares the fixed long-burst/cost model rather than every interactive workload.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multics

Segments, files, paging, and selective sharing combine with ring enforcement. Rings and software layers provide different guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A computer utility with controlled sharing

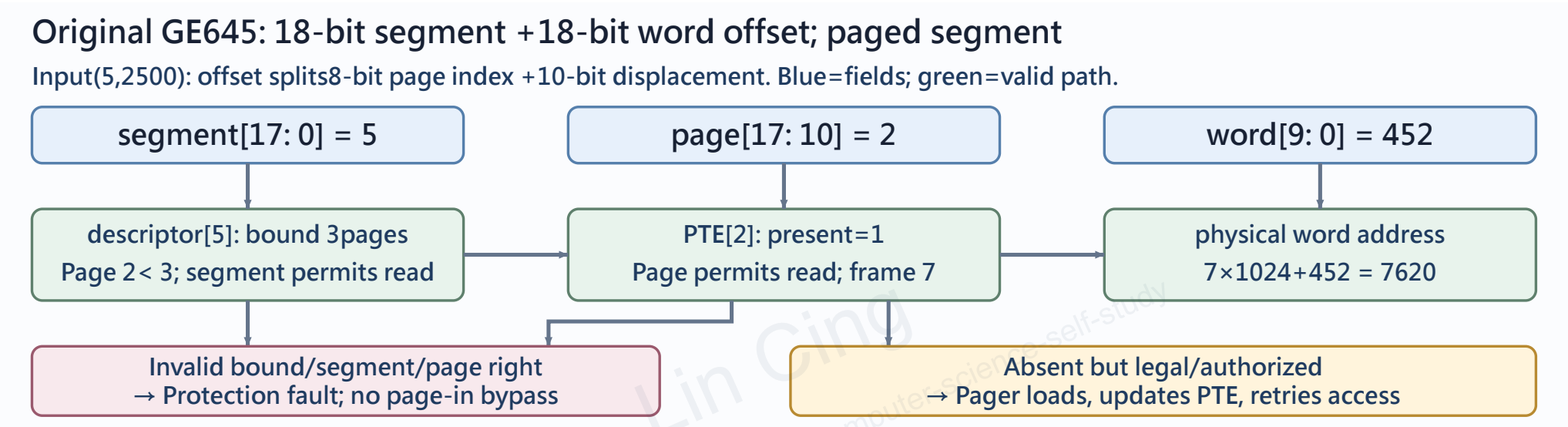
Multics targets remote terminals, continuous service, growing capacity, and selective sharing. Hierarchical files, segments, paging, rings jointly support those goals rather than one vague early-large-OS label.

Fix original GE645 hardware:18-bit segment number and18-bit word offset, each0…$2^{18}-1$. A 1024-word page uses10 offset bits, leaving8 page-index bits. Descriptor bounds can be smaller than field capacity. Original design.

Segment5, offset 2500=2 • 1024+452:first inspect descriptor, bound, rights, then segment page 2. Frame7 gives core word address7620. Segment identity is independent of physical frame.

Here segment 5 is paged, bound 3 pages, and segment/page permit read. Page 2<3 and presentPTE2→frame 7 allow access. Invalid bound/permission is a protection fault that page-in cannot bypass. Only an authorized absent page invokes pager, mapping update, retry.

A computer utility with controlled sharing (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rings versus layers

Rings enforce privilege/entry rules;THE layers organize abstraction/dependency. Modules need not each own a ring, and drawing layers proves no protection boundary.

Resolving a segment name still requires object access lists, execution context, entry rules. Shared code/private writable state need mapping/protection contracts. The developers'1972 retrospective describes integration exposing unexpected path costs and iterative redesign. Useful concepts do not prove the first interface partition efficient;smaller successors need not retain every mechanism. Retrospective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IBM OS/360

Family compatibility does not erase MFT/MVT differences. Later virtual memory and CP/CMS add distinct abstractions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compatibility and allocation variants

System/360 sought common software/tools across capacities. OS/360 MFT uses fixed regions;MVT variable regions. One variant does not specify the entire family.

A 256 KiB example has four64 KiB regions and jobs50,50,70. Two50 jobs waste28 KiB internally. Remaining128 total cannot fit70 into one64 region. This is the chosen configuration, not all possible OS/360 layouts.

Variable holes50,30,40 total 120 but lack a contiguous70, an external-fragmentation example. Moving/compacting requires relocation support/cost;load-time-bound addresses cannot be arbitrarily changed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Later virtual memory and CP/CMS

System/370-era OS/VS/MVS uses new VM hardware rather than original360 paging semantics. CP/67 provides VMs;CMS is a single-user system within them. CP supplies multi-CMS isolation rather than CMS itself becoming a shared multiuser kernel.

Two guests can both see address1000 while VMM maps their memory views to different host resources. Mediate guest privilege;guest supervisor status does not authorize arbitrary host writes. Family compatibility, relocation, VM isolation, and interaction are distinct. JCL describes batch resources/steps;option count proves neither correctness nor a fixed performance ratio.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TOPS-20

TENEX/TOPS-20 combines command assistance and memory support. Syntax usability does not replace object authorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interaction and virtual memory

In the book's history, BBN added paging to PDP-10-related hardware and developed TENEX. DEC later acquired rights and formed DECSYSTEM-20/TOPS-20. Command completion/help reduces syntax-memory burden;virtual memory manages addresses/residence. Shell suggestions do not authorize page-table changes, and paging does not infer desired commands.

A machine-like private execution view with many simultaneous users still needs CPU scheduling, terminal I/O, file rights, reclamation. Faster CPUs alone do not provide interaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Usability and security complement one another

In an explicit toy COPY source destination parser, valid syntax creates a request. Execution still checks source read and destination-parent/file write contracts. Autocomplete success does not grant access.

Checking destination then later writing its pathname can race name replacement, as in Chapter 11 TOCTOU. Protected references, clear errors, and understandable interaction must work together. This toy illustrates responsibility, not literal TOPS-20 syntax.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CP/M and MS-DOS

Small-platform compatibility differs from modern protected kernels. A shared-address-space convention does not confine untrusted code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resource limits and isolation

CP/M supplies early microcomputer commands/program/device services. MS-DOS follows an IBM-PC single-user/compatibility path. Their protection differs from later multiuser systems;they are not merely smaller NT/Linux.

Counts such as64 KiB/640 KiB must distinguish CPU addressability, conventional layout, OS reservations, and extensions. A common program region is not every version's absolute physical limit.

Without enforced memory isolation, programs can overwrite system data despite APIs instructing them to use services. Hostile programs need not follow conventions. Hardware privilege plus trusted mediation is required.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A shared-space counterexample

Toy RAM[0,1024) has monitor[0,256), job[256,1024), without hardware bounds. Store100 corrupts monitor even if correct libraries never do so. This is not an actual DOS layout.

Protected base256, limit768 validates0≤a<768 then translates256+a. Addresses−1/768 fail;0/767 reach256/1023. Users cannot modify base/limit. This proves range confinement, not progress against infinite loops, which needs a timer, or shared-file policy. One hardware feature supplies one boundary rather than a full modern OS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Macintosh and Windows

GUI design does not determine preemption, memory protection, or driver boundaries.
Cooperative/preemptive progress rests on different premises.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Desktop interaction versus kernel guarantees

Early Macintosh/Windows made desktop interaction central;later generations added fuller memory, multitasking, networking. Specify generation. Windows 7 here is NT, not early DOS-dependent Windows.

Windows, mouse, menus are interaction models;preemption, address protection, compatibility, drivers are separate. Two windows prove neither parallel processes nor isolated address spaces. A blocked UI thread can freeze a GUI while kernel scheduling continues. Trace event, queue, handler, and awaited service.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cooperative and preemptive progress

On one CPU, A loops forever without yield while B is ready. If switching requires yield/block, B may never run. With serviceable timer and a scheduler giving both slices, preemption runsB at a slice boundary.

If A instead loops unbounded in a nonpreemptible kernel lock region, user slices alone cannot guarantee progress. Include kernel/IRQ bounds. This toy compares protocols without classifying every desktop generation identically. Even a scheduledB may wait for A's global resource;preemption does not eliminate deadlock/priority inversion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach

Mach2.x and3 place BSD differently. Separate task/thread/port authority from copy/mapping ownership.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Versioned abstractions

Mach builds on Accent-related research with tasks/address spaces, threads, IPC, VM. A task owns address space/port rights;threads execute and share their task's memory.

Model	BSD placement	Boundary
Mach2.x/2.5	Much remains in kernel	File service need not cross a user server
Mach3 microkernel	UNIX services in user servers	Controlled IPC across address spaces

Kernel-checkable port rights authorize messages;equal integers do not transfer rights across tasks. Shared memory also exists, so not all exchange requires byte-copy messages.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A fixed IPC cost and isolation model

ClientC sends32 B to user serverS, which replies4096 B. Each direction copies user→kernel→other user twice, without shared-page/out-of-line optimization.

Path	Mode transitions	Address-space changes	Copy bytes
C user→K→S user	2	1	64
S user→K→C user	2	1	8192
Total	4	2	8256

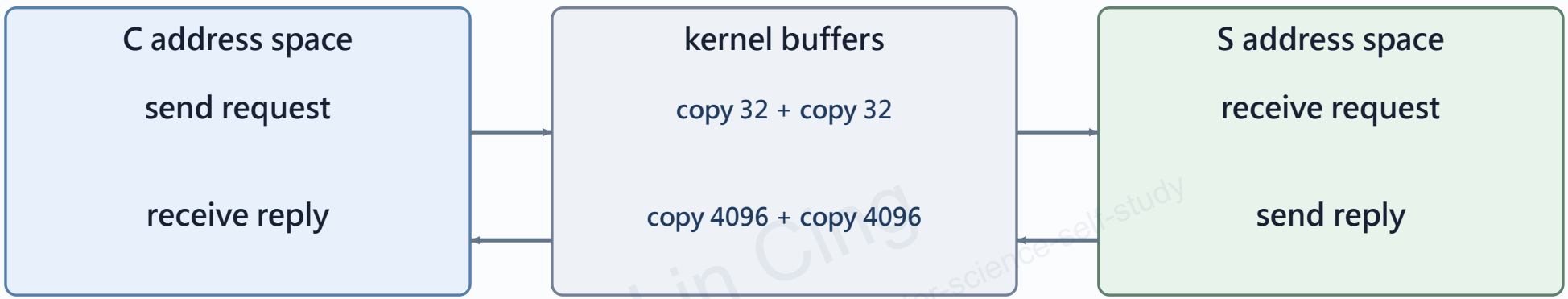
Address protection confines direct corruption from server crash, but client still needs errors, timeouts, reconstruction. Smaller kernel does not prove universal availability. IPC/VM integration can avoid copies while adding mapping, rights, page lifetime, COW costs rather than becoming free.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed IPC cost and isolation model (continued)

Mach3 copy model: 32 B request +4096 B reply; 8256 B copied

Blue=client C; green=server S; gray=kernel. Two copies/direction, no shared-page optimization.



Controlled kernel transfer: 4 mode transitions,2 address-space changes. Check port rights and buffer lifetime.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Other systems and unequal processor roles

The book also discusses Burroughs MCP's systems language, segmentation, multiprocessing, and CDC6600 coordination among processing activities. Multiple processors need not be symmetric peers. Identify roles, memory authority, I/O responsibility before applying an SMP scheduler.

High-level languages improve expression, type/module checking, maintenance, while synchronization, ABI, hardware control still need correctness. Assembly does not logically imply failure. Language choice changes inspectable conditions rather than replacing contracts.

Compatibility ecosystems, hardware costs, deployment organizations also affect survival. No single feature here explains commercial success or supplies current-market evidence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mechanisms across cases

Problem	Case	Required conditions
CPU/slow-I/O overlap	Monitors/spools	Queues, buffers, ownership, overlappable work
Nonresident information	Atlas	Mapping, restartable fault, total victim selection
Scattered/shared code	XDS-940	Relocation, RO code, private mutable state
Decomposed correctness	THE	Acyclic dependencies, layer contracts
Policy above nucleus	RC4000	Message ownership, pool limits, wait relationships
Interaction/long-burst tradeoff	CTSS	Feedback, promotion, swap-cost rules
Selective sharing/naming	Multics	Separate segment, page, protection
Service isolation	Mach3	IPC rights, address spaces, failure handling

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General proofs beyond finite demonstrations

A trace exposes missing t=0 branches, word/byte confusion, or duplicate buffer allocation. General claims require rules covering all legal states.

Atlas selects a victim from every nonempty finite frame set:if branch 1 applies, use it;otherwise if any nonzero t exists, a finite maximum score exists;otherwise branch 3 chooses maximalT. This proves total selection, not minimum faults.

Buffer conservation holds initially and under each atomic ownership transition. Appearing simultaneously free and awaiting reply violates that invariant. Layer induction requiresCi to depend only on lower contracts while preserving them;cycles/uncontrolled hidden-state mutation invalidate it.

These prove total selection, resource safety, and compositional correctness, respectively. None alone establishes an entirely reliable OS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

- Silberschatz, Galvin, and Gagne,*Operating System Concepts*,9th edition, Chapter 20, printed pages 887–904. Historical cases retain their configurations. Numerical models, questions, SVGs are original;word counts are not byte counts.
- Kilburn1962 one-level store, pp231–232 supplies the three learning branches, including all-t=0. Historical intervals predict rather than implementOPT.
- Dijkstra EWD196 defines THE levels/information segments versus storage pages without implying independent address spaces.
- GE645 design specifies18-bit segment and18-bit word offset, split8+10 for 1024-word pages, not later6180 pointer formats.
- Multics first seven years,1972 describes utility goals/iteration rather than current markets.
- Tom Van Vleck on CTSS supplies7094/36-bit/compatibility context. Fixed q_i=2^i cases are not historical per-tick reconstructions.

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study